

Project Proposal/Synopsis

2023-24

Team Details

S.No	Roll Number	Name	Email	Contact number
1	[Roll No 1]	Sagar [Full Name] (TL)	sagar[email]@domain.com	+91-XXXXXXXXXX
2	[Roll No 2]	[Team Member 2 Name]	member2[email]@domain.com	+91-XXXXXXXXXX
3	[Roll No 3]	[Team Member 3 Name]	member3[email]@domain.com	+91-XXXXXXXXXX

Project Details

Title: Offline UPI: Secure Multi-Hop BLE Mesh Routed Payment and Settlement Engine

Technologies to be used:

- **Backend Framework & APIs:** Python 3, Flask Web Framework, RESTful API architecture.
- **Database & Ledger:** SQLite3 (upi.db) with ACID Transactions and Optimistic Version Locking.
- **Frontend & Dashboard:** HTML5, Modern CSS3 (Dark Theme UI), Vanilla JavaScript (Fetch API).
- **Cryptography & Security:** Python cryptography library (RSA-2048-OAEP, AES-256-GCM, SHA-256 Hashing).
- **Distributed Mesh Simulation:** Python Store-and-Forward Gossip Engine, Hop Decrement, Opportunistic Bridge Ingestion.
- **Testing & Quality Assurance:** Python standard unittest suite (Crypto roundtrip, Tamper rejection, Exact-once settlement races).

Brief Description of the Project (Point-wise):

- **Problem Statement:** Standard UPI payments fail completely when users are in offline dead-zones (underground basements, parking lots, transits, remote rural locations, or disaster relief zones) due to absence of active mobile data or Wi-Fi.
- **Multi-Hop Mesh Gossip Architecture:** This project implements an offline peer-to-peer store-and-forward mesh protocol. An offline sender creates an encrypted payment packet and broadcasts it over simulated Bluetooth Low Energy (BLE) gossip. Intermediate bystander phones act as passive relay nodes, passing the packet hop-by-hop without requiring internet until an opportunistic Bridge Node with 4G/Wi-Fi uploads it to the backend.
- **Zero-Trust Hybrid Cryptography:** Intermediary phones are untrusted and cannot read or tamper with transaction details. The payment instruction is encrypted using **AES-256-GCM**, and the session AES key is enveloped using the Server's **RSA-2048-OAEP** public key. Any alteration in ciphertext invalidates the 128-bit authentication tag, causing immediate cryptographic rejection at the server.
- **Idempotency & Deduplication Engine:** If multiple bridge nodes upload the same broadcasted packet concurrently, SQLite's primary key constraint on SHA-256 ciphertext hash acts as an atomic putIfAbsent claim. The first upload settles (SETTLED); subsequent duplicate uploads are safely discarded (DUPLICATE_DROPPED) preventing double-spending.
- **Freshness & Nonce Verification:** Each packet includes a cryptographically generated UUIDv4 nonce and timestamp. A strict 24-hour freshness sliding window prevents replay attacks.
- **ACID Ledger Settlement in SQLite:** Upon successful decryption and validation, an atomic SQLite transaction debits the sender's account, credits the receiver, increments version locks, and logs an immutable audit trail.
- **Interactive Web Dashboard:** A clean Flask-powered web dashboard enables examiners to live-test the 4-step workflow: creating offline payments, stepping through BLE gossip hops, uploading through the 4G bridge, injecting tamper attacks, and inspecting live SQLite tables.

Whether compared with any existing system: (Give URL /link or citation)

Yes. Compared with:

1. **NPCI UPI 123PAY (*99# USSD/IVR):** Requires active telecom cellular carrier service for voice/SMS; fails in RF-shielded basements and zero-signal deadzones (<https://www.npci.org.in/what-we-do/upi-123pay/product-overview>).
2. **RBI Offline Payment Framework for Small Value Transactions:** Limited to proximity NFC / hardware-secure-element proximity wallets (<https://www.rbi.org.in/Scripts/NotificationUser.aspx?Id=12215&Mode=0>).
3. **Delay-Tolerant Networking (DTN / RFC 4838):** Multi-hop store-and-forward gossip protocols across challenged networks (Fall, K., RFC 4838).

Whether discussed with any Faculty Member (NAME):

Yes. Discussed with [Prof. / Dr. Faculty Mentor Name], Department of Computer Science & Engineering.

Whether proposed work is extension of internship work:

No. It is an original, self-initiated Capstone Research & Engineering Project.

Any previous work with proposed technologies:

Yes. Team members have completed coursework and practical lab projects in Python programming, Flask web frameworks, SQLite relational databases, and Applied Cryptography.

Whether crude DFD/ ERD are prepared:

Yes. Level 0, Level 1, and Level 2 Data Flow Diagrams (DFDs) along with comprehensive Entity-Relationship Diagrams (ERDs) detailing Account, MeshPacket, IdempotencyClaim, and Transaction models are prepared.

Planning Details (July-23 to Apr-24)

Phase	From	To
Literature Survey	17 July 2023	31 August 2023
Design (Architecture, DFD, Crypto Schemas)	01 September 2023	15 October 2023
Implementation-1 (Python Cryptography & SQLite DB Setup)	16 October 2023	30 November 2023
Implementation-2 (Mesh Simulation Engine & Flask Ingestion API)	01 December 2023	20 January 2024
Implementation-3 (Idempotency Claim Engine & Web Dashboard)	21 January 2024	10 March 2024
Testing (Concurrency, Tamper Injection & Unittest Suite)	11 March 2024	10 April 2024
Submission	15 April 2024	15 April 2024

WORK DISTRIBUTION PLAN

PROJECT ID: - [Pending Acceptance] **TEAM LEADER:** - Sagar [Full Name] **PROJECT TITLE:** - Offline UPI: Secure Multi-Hop BLE Mesh Routed Payment & Settlement Engine

S.N o	MODULE NAME(S)	FUNCTIONALITIES	TECHNOLOGIES USED	TEAM MEMBER
1.	Hybrid Cryptographic & Key Management Engine	• Server RSA-2048 keypair generation and management. • Hybrid RSA-2048-OAEP key wrapping & AES-256-GCM cipher engine. • 128-bit authentication tag verification & tampering rejection. • SHA-256 packet hashing and nonce generation.	Python 3, cryptography, RSA-2048, AES-256-GCM, SHA-256	Sagar (TL)
2.	Bridge Ingestion & Idempotent Settlement Service	• Flask Bridge ingestion API (/api/bridge/ingest). • Atomic SQLite primary key claim cache for deduplication. • Settlement resolver ensuring exact-once balance debit/credit. • Account balance manager with optimistic version locks.	Python 3, Flask, SQLite3, ACID Transactions	Sagar (TL)
3.	P2P BLE Mesh Simulation & Gossip Routing Engine	• Virtual device topology state modeling (offline phones vs. bridge node). • Store-and-forward Delay Tolerant Gossip algorithm. • Packet Time-To-Live (TTL) decrement & multi-hop counter. • Opportunistic bridge synchronization and flush endpoints.	Python 3, Distributed Systems Simulation Logic	[Team Member 2]
4.	Client-Side Packet Injection & Demo Workflow	• Offline simulated sender device packet assembler. • PIN hashing and client payment instruction validation. • Interactive 4-step execution flow controller (Create → Gossip → Connect → Flush). • Mesh topology state reset and test-run initialization.	Python 3, Flask Endpoints, JavaScript	[Team Member 2]
5.	Interactive Dashboard & Mesh Visualizer UI	• Real-time graphical mesh topology component. • Visual device status cards with held packet counters and live connection badges. • Interactive simulation control panel. • Dark-mode responsive UI and navigation bar with live telemetry logs.	HTML5, Modern CSS3, Vanilla JavaScript (Fetch API)	[Team Member 3]

6.	Ledger Audit Interface & Automated Test Suite	• Live double-entry Bank Accounts and Transaction Ledger tables. • Real-time settlement status badges (SETTLED, DUPLICATE_DROPPED, INVALID). • Automated Python unittest suite covering crypto symmetry, tampering, and concurrent bridge upload races.	Python unittest, SQLite3, HTML5/JS	[Team Member 3]
----	--	---	------------------------------------	------------------------

Remember:

- Permanent PROJECT ID would be provided only after acceptance of proposal.
- If more than one person is allocated on a module, it should be mentioned who is dealing with which functionalities of the module.
- Create this distribution PLAN in detail.
- Distribution of workload should be balanced among team members.